

Лабораторная работа №7. Адаптивная верстка: Flexbox

Цель работы: освоить современные технологии CSS-разметки — Flexbox.

Адаптивная верстка — это подход к созданию веб-страниц, при котором дизайн и контент автоматически подстраиваются под размер экрана устройства (компьютер, планшет, смартфон).

Flexbox — одномерная система компоновки для размещения элементов в строку или столбец.

Шаг 1. Создание рабочей директории

1. Откройте **Visual Studio Code**. Выберите **File** → **New Window** (Ctrl + Shift + N).
2. Нажмите **Ctrl + `** чтобы открыть терминал VS Code.
3. Перейдите в папку с документами:

cd Documents/

4. Создайте директорию для лабораторной работы:

mkdir Lab7_Adaptive_FIO (где *FIO* — ваша фамилия)

5. Перейдите в созданную директорию:

cd Lab7_Adaptive_FIO

6. Создайте файлы:

touch index.html styles.css README.md

- **index.html** — для HTML-разметки
- **styles.css** — для CSS-стилей
- **README.md** — для описания

7. Создайте папку для скриншотов:

mkdir img

Шаг 2. Подключение рабочего каталога в VS Code

1. Откройте **Explorer** (Ctrl + B).
2. Нажмите **Open Folder**.
3. Выберите папку **Lab7_Adaptive_FIO** и откройте её.

Шаг 3. Инициализация Git-репозитория

1. Инициализируйте репозиторий:

git init -b main

2.Добавьте файлы в индекс:

git add .

3.Создайте первый коммит:

git commit -m "Initial commit"

Шаг 4. Подключение к GitHub

1. Создайте новый **public**-репозиторий на GitHub:

Lab7_AdaptiveFlexbox

Без **README**, **.gitignore** и **License**.

2. Привяжите локальный репозиторий:

git remote add origin <URL_вашего_репозитория>

git push -u origin main

Практическое задание

1. Сделайте скриншот **Git Bash** с выполненными командами подключения и пуша.

2. Сохраните файл в папку **img**:

gitPushLab7_FIO.png

3. Выполните в терминале:

git add .

git commit -m "Added push screenshot"

git push

Шаг 5. Создание базовой структуры

Откройте файл **index.html** и создайте базовую разметку (используйте **! + Enter**):

```
<!DOCTYPE html>
<html lang="ru-RU">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Адаптивная верстка: Flexbox и Grid</title>
    <link rel="stylesheet" href="styles.css" />
  </head>
  <body>
    <h1>Лабораторная работа №7</h1>
    <p>Изучаем Flexbox и Grid</p>
  </body>
</html>
```

Важно! Обратите внимание на мета-тег:

<meta name="viewport" content="width=device-width, initial-scale=1.0">

Что он делает:

- **width=device-width** — ширина страницы равна ширине устройства
- **initial-scale=1.0** — начальный масштаб 100%

Без этого тега адаптивная верстка **не будет работать** на мобильных устройствах!

Теория. Введение в Flexbox

Flexbox (Flexible Box Layout) — это модуль CSS для создания гибких одномерных раскладок. Он идеально подходит для:

- Выравнивания элементов по горизонтали или вертикали
- Распределения свободного пространства
- Создания навигационных меню
- Центрирования элементов

Основные концепции

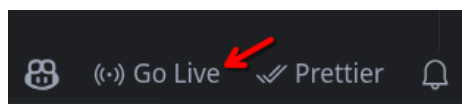
Flex-контейнер — родительский элемент с `display: flex`

Flex-элементы — дочерние элементы внутри контейнера

1. Обновим **index.html**. Мы добавим **контейнер** и несколько **элементов**, с которыми будем экспериментировать:

```
<p>Изучаем Flexbox и Grid</p>
<div class="container">
  <div class="item">1</div>
  <div class="item">2</div>
  <div class="item">3</div>
</div>
</body>
```

2. Запустите вашу страницу через **Live Server**, чтобы наблюдать изменения:



Сейчас у нас отображается страница в следующем виде:

Лабораторная работа №7

Изучаем Flexbox и Grid

1
2
3

3. Добавим базовые стили в файл **style.css**.

Начнём с **простых визуальных стилей**, чтобы изменения были хорошо заметны:

```
body {
  font-family: Arial, sans-serif;
}

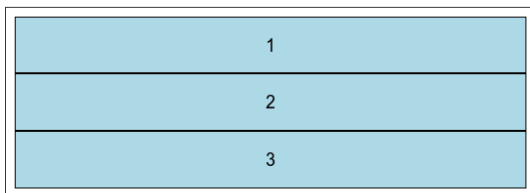
.container {
  border: 2px solid #333;
  padding: 10px;
}

.item {
  background-color: lightblue;
  border: 1px solid #000;
  padding: 20px;
  font-size: 20px;
  text-align: center;
}
```

Смотрим на изменения:

Лабораторная работа №7

Изучаем Flexbox и Grid



На этом этапе элементы идут столбцом, потому что Flexbox ещё не включён.

5.1. Делаем flex-контейнер

Добавляем в начале файла **style.css**:

```
.container {
  display: flex; /* Создаем flex-контейнер */
}
```

Что видим в браузере:

- Элементы выстраиваются в одну строку
- Это поведение flex-direction: row по умолчанию

Лабораторная работа №7

Изучаем Flexbox и Grid



5.2. Свойство `flex-direction`

Пример 1: строка (по умолчанию)

По умолчанию у нас используется свойство `flex-direction: row`. Мы можем в этом убедиться, обновив в `style.css`:

```
.container {  
  display: flex; /* Соз  
+ flex-direction: row;  
}
```

Пример 2: колонка

Теперь обновим `flex-direction` в `style.css`:

```
.container {  
  display: flex; /* Создае  
  flex-direction: column;  
}
```

Визуальные изменения (элементы сверху вниз):

Лабораторная работа №7

Изучаем Flexbox и Grid

1
2
3

Проверьте другие возможные значения для `flex-direction`:

- **row** — элементы в строку (→)
- **row-reverse** — элементы в строку справа налево (←)
- **column** — элементы в столбец (↓)
- **column-reverse** — элементы в столбец снизу вверх (↑)

Практическое задание

1. Сделайте 4 скриншота страницы разных поведений **flex-direction**.
2. Сохраните файлы в папку **img**:

flex-direction_row_Lab7_FIO.png

flex-direction_column_Lab7_FIO.png

flex-direction_row-reverse_Lab7_FIO.png

flex-direction_column-reverse_Lab7_FIO.png

5.3. justify-content — выравнивание по главной оси

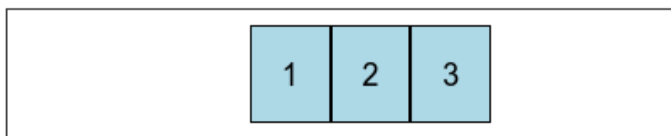
Добавьте в `style.css`:

```
.container {  
  display: flex; /* Создаем  
  flex-direction: row;  
  justify-content: center;  
}
```

Визуальные изменения:

Лабораторная работа №7

Изучаем Flexbox и Grid



Проверьте другие возможные значения для *justify-content*:

- **flex-start** — в начале (по умолчанию)
- **flex-end** — в конце
- **center** — по центру
- **space-between** — равномерно, первый и последний у краёв
- **space-around** — равномерно, с отступами по краям
- **space-evenly** — равномерно, одинаковые отступы везде

Практическое задание

1. Сделайте 6 скриншотов страницы разных поведений **justify-content**.
2. Сохраните файлы в папку **img**:

justify-content_flex-start_Lab7_FIO.png

justify-content_flex-end_Lab7_FIO.png

justify-content_center_Lab7_FIO.png

justify-content_space-between_Lab7_FIO.png

justify-content_space-around_Lab7_FIO.png

justify-content_space-evenly_Lab7_FIO.png

5.4. align-items — выравнивание по поперечной оси

Добавим высоту контейнеру для наглядности:

```
.container {  
  display: flex; /* Созда  
  flex-direction: row;  
  height: 200px;  
  align-items: stretch;  
}
```

Проверьте возможные значения для *align-items*:

- **stretch** — растягивает элементы (по умолчанию)
- **flex-start** — в начале
- **flex-end** — в конце
- **center** — по центру
- **baseline** — по базовой линии текста

Практическое задание

1. Сделайте 5 скриншотов страницы разных поведений **align-items**.
2. Сохраните файлы в папку **img**:

align-items_stretch_Lab7_FIO.png

align-items_flex-start_Lab7_FIO.png

align-items_flex-end_Lab7_FIO.png

align-items_center_Lab7_FIO.png

align-items_baseline_Lab7_FIO.png

5.5. flex-wrap — перенос элементов

Добавим дополнительно **100 элементов** в **index.html** в тег **<body>**.

Лайфхак: Быстрое создание множества элементов с Emmet. Вместо ручного копирования или написания каждого элемента, используем Emmet — мощный инструмент, встроенный в VS Code.

1. Откройте файл **index.html**
2. Установите курсор внутри тега **<body>**, внутри контейнера **container**
3. Введите команду:

.item{\$}*100

```

<body>
  <h1>Лабораторная работа №7</h1>
  <p>Изучаем Flexbox и Grid</p>
  <div class="container">
    <div class="item">1</div>
    <div class="item">2</div>
    <div class="item">3</div>
    .item{$}*100
  </div>
</body>

```

4. Нажмите клавишу Tab

Разбор команды:

- **.item** — создаёт элемент с классом item
- **{\$}** — добавляет текст с нумерацией внутри элемента
- ***100** — умножает элемент 100 раз

Теперь если мы посмотрим на нашу страницу в браузере, то увидим, что у нас отсутствуют переносы:

Лабораторная работа №7

Изучаем Flexbox и Grid

1	2	3	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	:
---	---	---	---	---	---	---	---	---	---	---	---	----	----	----	----	----	----	----	----	----	----	----	---

Это эквивалентно следующей записи (напишите в **style.css**):

```

.container {
  display: flex;
  flex-wrap: nowrap;
}

```

Проверьте возможные значения для переноса элементов:

- **nowrap** — без переноса (по умолчанию)
- **wrap** — перенос на новую строку
- **wrap-reverse** — перенос в обратном направлении

Практическое задание

1. Сделайте 3 скриншота страницы разных поведений **flex-wrap**.
2. Сохраните файлы в папку **img**:

flex-wrap_nowrap_Lab7_FIO.png

flex-wrap_wrap_Lab7_FIO.png

flex-wrap_wrap-reverse_Lab7_FIO.png

5.6. gap — отступы между элементами

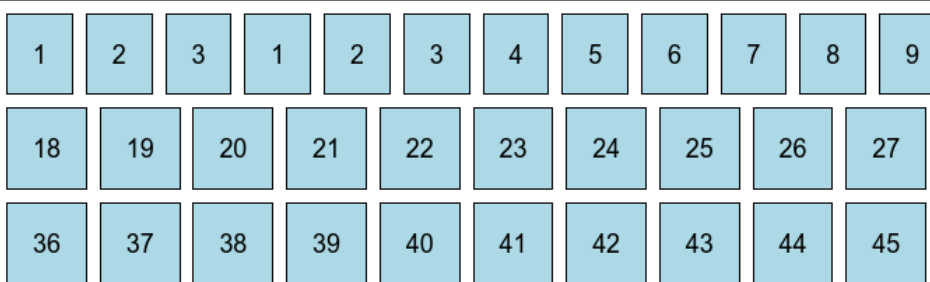
Добавьте дополнительные отступы:

```
.container {  
  display: flex;  
  flex-wrap: wrap;  
  + gap: 10px;  
}
```

Визуальное изменение:

Лабораторная работа №7

Изучаем Flexbox и Grid



Удобная современная замена *margin*.

Практическое задание

1. Сделайте скриншот страницы с отступами.
2. Сохраните файл в папку **img**:

gap_Lab7_FIO.png

5.7. Основные свойства элементов

5.7.1. flex-grow — растягивание элемента

Добавьте для контейнеров **item**:

```
.container {  
  display: flex;  
  flex-wrap: wrap;  
  gap: 10px;  
}  
  
.item {  
  flex-grow: 1;  
}
```

Элемент займёт свободное пространство.

Для лучшей демонстрации работы, измените масштаб страницы в браузере (**Ctrl + колесо мыши** или **Ctrl + =**).

90%

Лабораторная работа №7

Изучаем Flexbox и Grid

1	2	3	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35		
36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53		
54	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71		
72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89		
90	91	92	93	94	95	96	97	98	99	100									

200%

85	86	87	88	89	90	91
92	93	94	95	96	97	98
99			100			

Практическое задание

1. Сделайте 2 скриншота страницы при разном растягивании элементов.
2. Сохраните файлы в папку **img** (указывая используемый масштаб):

flex-grow_100_Lab7_FIO.png

flex-grow_350_Lab7_FIO.png

5.7.2. flex-basis — базовый размер (ширина) элемента

```
.item {  
  flex-basis: 150px;  
}
```

Визуальное изменение:

1	2	3
1	2	3

5.7.3. flex-shrink — сжатие элемента

5.7.4. flex — краткая запись

5.7.5. align-self — индивидуальное выравнивание

```
.item {  
  /* элемент не будет сжиматься */  
  flex-shrink: 0;  
  /* эквивалентно flex: 1 1 0 */  
  flex: 1;  
  /* переопределяет align-items для этого элемента */  
  align-self: flex-end;  
}
```

Практическое задание

Выполните в терминале:

git add .

git commit -m "Learn and practice Flexbox"

git push

Шаг 6. Практика с Flexbox

В этом шаге мы рассмотрим три практических примера Flexbox:

1. Горизонтальное навигационное меню
2. Карточки товаров
3. Центрирование элемента

Все изменения смотрим в реальном времени через Live Server.

6.1. Создание нового HTML-файла

6.1.1. Создаём файл

1. Через терминал создайте файлы **flexbox.html** и **flexbox.css**
touch flexbox.html flexbox.css
2. Откройте **flexbox.html** в редакторе
3. Введите **Emmet-команду !** и нажмите **Enter**

```
<!DOCTYPE html>  
<html lang="en">  
  <head>  
    <meta charset="UTF-8" />  
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />  
    <title>Document</title>  
  </head>  
  <body></body>  
</html>
```

Emmet автоматически создаёт структуру HTML

6.1.2. Исправляем язык и заголовок

Измените вручную:

```
<html lang="ru-RU">
```

```
<title>Flexbox Examples</title>
```

6.1.3. Подключаем CSS-файл

Внутри **<head>** добавьте строку:

```
<link rel="stylesheet" href="flexbox.css" />
```



Emmet-подсказка: Начните вводить **link** → выберите **link:css**

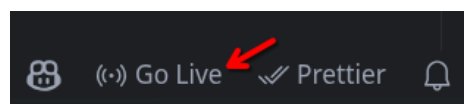


6.2. Заголовок страницы

Внутри **<body>** добавьте:

```
<body>
+<h1>Примеры Flexbox</h1>
</body>
```

Запустите вашу страницу через **Live Server**, чтобы наблюдать изменения:



6.3. Пример 1. Навигационное меню (Flex-контейнер)

6.3.1. Создаём секцию

В **<body>** ниже заголовка напишите:

```
<body>
  <h1>Примеры Flexbox</h1>
  +<section>
    <h2>Пример 1: Навигационное меню</h2>
  </section>
</body>
```

Пояснение: **section** логически группирует контент на странице.

6.3.2. Добавляем меню через Emmet

1. Внутри `<section>` введите:

nav.navbar>a*4

2. Нажмите **TAB**

```
<section>
  <h2>Пример 1: Навигационное меню</h2>
  +nav.navbar>a*4 TAB
</section>
```

3. Отформатируем немного вручную:

```
<h2>Пример 1: Навигационное меню</h2>
<nav class="navbar">
  <a href=""></a>
  <a href=""></a>
  <a href=""></a>
  <a href=""></a>
</nav>
</section>
```

4. Теперь добавим текст ссылкам:

```
<nav class="navbar">
  <a href="#">Главная</a>
  <a href="#">О нас</a>
  <a href="#">Услуги</a>
  <a href="#">Контакты</a>
</nav>
```

Смотрим визуально нашу страницу в браузере:

Примеры Flexbox

Пример 1: Навигационное меню

[Главная](#) [О нас](#) [Услуги](#) [Контакты](#)

Практическое задание

1. Сделайте скриншот страницы в браузере.
2. Сохраните файл в папку **img**:

flexbox_html_Lab7_FIO.png

6.3.3. Подключаем стили в CSS-файл

В файл **flexbox.css** добавьте базовые стили:

```
* {  
  margin: 0;  
  padding: 0;  
  box-sizing: border-box;  
}
```

Что это делает:

- Убирает стандартные отступы браузера
- Упрощает расчёт размеров элементов

Смотрим на визуальные изменения страницы в браузере:

Примеры Flexbox

Пример 1: Навигационное меню

[Главная](#) [О нас](#) [Услуги](#) [Контакты](#)

6.3.4. Делаем меню flex-контейнером

Добавьте в **flexbox.css**:

```
.navbar {  
  display: flex;  
  justify-content: space-around;  
  background-color: #333;  
  padding: 15px;  
}
```

Что видим в браузере:

Примеры Flexbox

Пример 1: Навигационное меню



- Ссылки выстроились в строку
- Равномерно распределились по ширине
- Flexbox работает

Практическое задание

1. Сделайте скриншот страницы в браузере.
2. Сохраните файл в папку **img**:

add_flexbox_Lab7_FIO.png

6.4. Пример 2. Карточки товаров

6.4.1 Создаём структуру карточек

Добавьте новую секцию в **flexbox.html**:

```
9  <body>
10  <h1>Примеры Flexbox</h1>
11  <!-- Пример 1: Горизонтальное меню -->
12  <section> ...
20  </section>
21  <!-- Пример 2: Карточки -->
22  <section>
23    <h2>Пример 2: Карточки товаров</h2>
24    + <div class="cards"></div>
25  </section>
26 </body>
```

6.4.2. Добавляем карточки через Emmet

Внутри **.cards** используйте:

div.card*3>h3+p+button

Нажмите **TAB**

```
<h2>Пример 2: Карточки товаров</h2>
<div class="cards">
  <div.card*3>h3+p+button TAB
</div>
```

Результат:

```
<!-- Пример 2: Карточки -->
<section>
  <h2>Пример 2: Карточки товаров</h2>
  <div class="cards">
    <div class="card">
      <h3></h3>
      <p></p>
      <button></button>
    </div>
    <div class="card">
      <h3></h3>
      <p></p>
      <button></button>
    </div>
    <div class="card">
      <h3></h3>
      <p></p>
      <button></button>
    </div>
  </div>
</section>
```

Добавьте описание товара:

```
<div class="card">
  <h3>Товар 1</h3>
  <p>Описание товара</p>
  <button>Купить</button>
</div>
```

(Аналогично для остальных карточек)

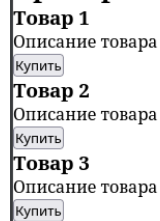
Визуальное отображение в браузере:

Примеры Flexbox

Пример 1: Навигационное меню



Пример 2: Карточки товаров

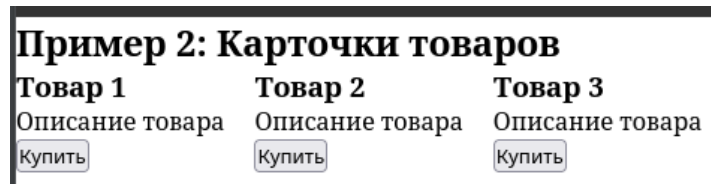


6.4.3. Делаем карточки flex-контейнером

В `flexbox.css` добавьте:

```
.cards {
  display: flex;
  gap: 20px;
  flex-wrap: wrap;
}
```

Результат:



- Карточки стоят в ряд
- При уменьшении окна переносятся на новую строку

Практическое задание

1. Сделайте скриншот страницы в браузере.
2. Сохраните файл в папку **img**:

add_flexbox_cards_Lab7_FIO.png

6.4.4. Настраиваем поведение карточек

В `flexbox.css` добавьте:

```
.card {
  flex: 1 1 250px;
  background-color: #f0f0f0;
  padding: 20px;
  border-radius: 10px;
  display: flex;
  flex-direction: column;
}
```

Объяснение flex: 1 1 250px:

- 1 — карточка может расти
- 1 — может сжиматься
- 250px — минимальная ширина

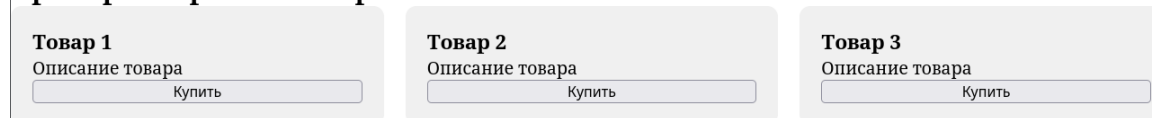
Результат:

Примеры Flexbox

Пример 1: Навигационное меню



Пример 2: Карточки товаров



Для лучшей демонстрации работы, измените масштаб страницы в браузере (**Ctrl + колесо мыши** или **Ctrl + =**).

Практическое задание

1. Сделайте скриншот страницы в браузере.
2. Сохраните файл в папку **img**:

add_flexbox_card_Lab7_FIO.png

6.4.5. Кнопка внизу карточки

В **flexbox.css** добавьте:

```
.card p {
  flex-grow: 1;
}

.card button {
  align-self: flex-start;
}
```

Что происходит:

- Текст растягивается
- Кнопка всегда прижата вниз

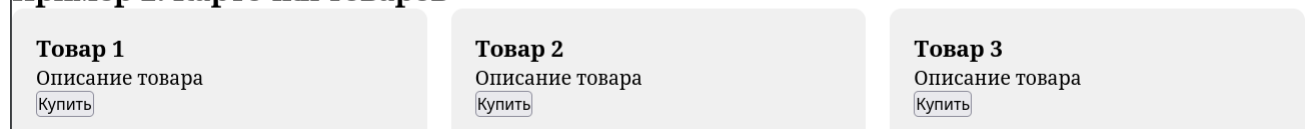
Результат:

Примеры Flexbox

Пример 1: Навигационное меню



Пример 2: Карточки товаров



Практическое задание

1. Сделайте скриншот страницы в браузере.
2. Сохраните файл в папку **img**:

flexbox_card_flex-start_Lab7_FIO.png

6.5 .Пример 3. Центрирование элемента

6.5.1. HTML-структура

Добавьте последнюю секцию:

```
42  <!-- Пример 3: Центрирование -->
43  <section>
44  <h2>Пример 3: Центрирование элемента</h2>
45  <div class="centered-box">
46  <div class="centered-content">
47  <h3>Я в центре!</h3>
48  </div>
49  </div>
50  </section>
51  </body>
```

Результат:

Пример 3: Центрирование элемента
Я в центре!

6.5.2. Центрирование через Flexbox

В **flexbox.css** добавьте:

```
.centered-box {
  display: flex;
  justify-content: center;
  align-items: center;
  height: 300px;
  background-color: #e0e0e0;
}
```

Результат:

Пример 3: Центрирование элемента

Я в центре!

Практическое задание

1. Сделайте скриншот страницы в браузере.
2. Сохраните файл в папку **img**:

flexbox_centered-content_Lab7_FIO.png

6.6. Общие стили страницы

6.6.1. Стили для body и секций

В файл **flexbox.css** добавьте:

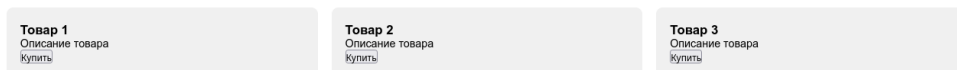
```
body {  
  font-family: Arial, sans-serif;  
  padding: 20px;  
}  
section {  
  margin-bottom: 40px;  
}  
h1, h2 {  
  margin-bottom: 20px;  
}
```

Результат:

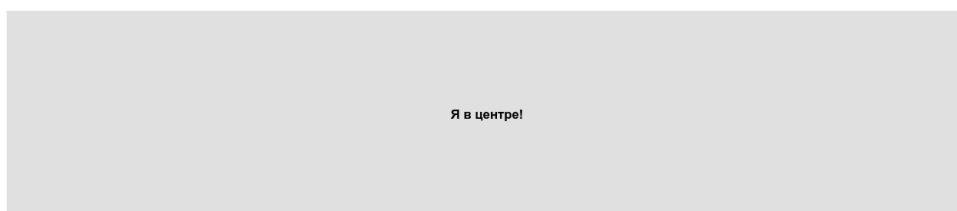
Пример 1: Навигационное меню



Пример 2: Карточки товаров



Пример 3: Центрирование элемента



Контент страницы стал аккуратнее, появились отступы между примерами.

6.7. Пример 1. Навигационное меню (доработка)

6.7.2. Стили ссылок меню

В файл **flexbox.css** добавьте:

```
.navbar a {  
  color: white;  
  text-decoration: none;  
  padding: 10px 20px;  
}
```

Результат:

Примеры Flexbox

Пример 1: Навигационное меню



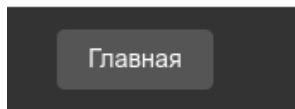
Ссылки стали читаемыми и удобными для навигации.

6.7.3. Эффект наведения на ссылки

В файл **flexbox.css** добавьте:

```
.navbar a:hover {  
  background-color: #555;  
  border-radius: 5px;  
}
```

Результат:



При наведении курсора ссылка подсвечивается.

6.8. Пример 2. Карточки товаров (доработка)

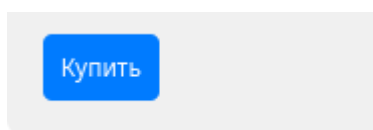
6.8.1. Оформление кнопки

Добавьте стили в кнопку:

```
.card button {  
  align-self: flex-start;  
  padding: 10px;  
  background-color: #007bff;  
  color: white;  
  border: none;  
  border-radius: 5px;  
  cursor: pointer;  
}
```

Результат:

Кнопка стала выглядеть как интерактивный элемент.



6.8.2. Эффект наведения на кнопку

Добавьте:

```
.card button:hover {  
  background-color: #0056b3;  
}
```

Результат:

Кнопка меняет цвет при наведении.



6.9. Пример 3. Центрирование элемента (доработка)

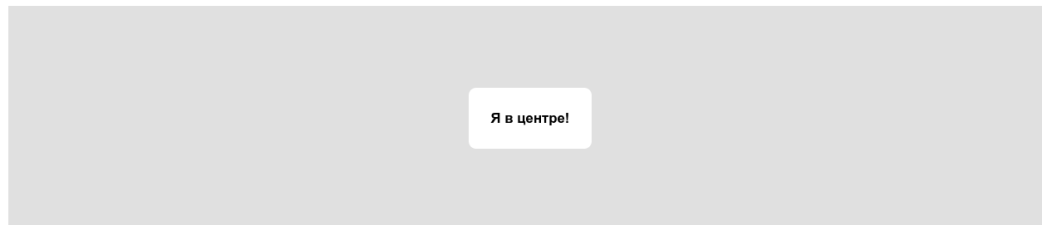
6.9.1. Оформление центрируемого блока

В **flexbox.css** добавьте:

```
.centered-content {  
  background-color: white;  
  padding: 30px;  
  border-radius: 10px;  
  box-shadow: 0 4px 10px rgba(0, 0, 0, 0.2);  
}
```

Результат:

Пример 3: Центрирование элемента



Контентный блок отчётливо выделяется и расположен строго по центру.

Практическое задание

1. Запустите страницу **flexbox.html** через **Live Server**.
2. Сделайте скриншот страницы в браузере со всеми примерами Flexbox.
3. Сохраните файл в папку **img**:

flexbox_practice_Lab7_FIO.png

Выполните в терминале:

git add .

git commit -m "Added Flexbox practice"

git push

Самостоятельное задание №1. Оформление README.md

1. Создайте подробное описание лабораторной работы в файле **README.md**.
2. Выполните в терминале:

git add .

git commit -m "Added complete README for Lab7"

git push

Самостоятельное задание №2

Сверстайте следующую страницу по образцу, с использованием только свойств Flexbox:

